Worked for 2 minutes ›

The current `poultry_geo` project is already a robust geospatial machine-learning pipeline. It slices massive NAIP GeoTIFFs into fixed-size tiles without padding, processes them in parallel, runs sliding-window inference, performs post-processing, and clusters individual detections into farms. The codebase includes modules for self-supervised learning (MIM for ViTs and SparK for CNNs), downstream fine-tuning with multiple YOLO variants, and GIS analytics. It is packaged as a CLI application with robust logging and stateful execution, and it has more than 140 unit tests. Evaluations show that the SparK-trained YOLO11l model achieves excellent precision and F1 scores for poultry-house segmentation and detection.

The MississippiFM specification proposes to build a new foundation model that leverages the known overlap between adjacent aerial tiles to align patch-level representations. The paper outlines three major contributions: (1) geographically aligned self-supervised learning using exact patch correspondence; (2) a sub-meter aerial foundation model pretrained on more than six million 60-cm tiles; and (3) a strict benchmark across states, counties, acquisition years, resolutions and tasks. It stresses that no new imagery should be generated—existing tiles, resolution and overlap must remain unchanged—and only a lightweight adjacency index mapping overlapping tiles needs to be added. The new training regime uses a DINOv3 ViT-B/16 backbone, four complementary objectives (global distillation, iBOT-style masked patch prediction, geographic overlap alignment, and Gram anchoring), and a pair-sampling strategy where half of each batch consists of overlapping tile pairs. It also recommends two-stage training with a low-resolution phase followed by a full 512×512 adaptation.

Where the project stands

The existing pipeline already accomplishes the ingestion, slicing, augmentation, self-supervised pretraining (MIM and SparK), downstream YOLO fine-tuning, sliding-window inference, and GIS post-processing. It has demonstrated strong performance on poultry-house detection and provided tools for county-level environmental analytics. However, it does not currently implement overlap-aware self-supervised learning, adjacency-index generation, a DINOv3 backbone or the specific multi-objective loss described in the MississippiFM plan.

Code updates needed to complete MississippiFM

1. **Create an adjacency index** – generate a metadata file that lists overlapping tile pairs and their pixel offsets. Use existing filenames or slicing logs to derive tile positions; do not reslice imagery.
2. **Implement an overlap-aware dataset** – extend the current `NAIPDataset` to sample overlapping tile pairs as well as independent tiles, returning patch-level overlap

masks and maintaining augmentation consistency.

3. **Add a DINOv3-based model** – wrap a ViT-B/16 backbone with heads for global distillation, masked patch prediction, overlap alignment, and Gram anchoring. Include projection heads and anchor weights as described.
4. **Revise the training loop** – incorporate the four losses with tunable weights, follow the pair-sampling proportions (50 % overlapping pairs, 25 % two views of one tile, 25 % independent tiles), and support two-stage training (crop resolutions and high-resolution adaptation). Use sequential execution and checkpointing to maintain the state.
5. **Expand evaluation scripts** – implement frozen feature probes and multi-task benchmarks (Agriculture-Vision, OpenEarthMap, Inria, DOTA) to assess representation quality beyond poultry houses, and add geographic transfer evaluations from Alabama to Georgia.
6. **Integrate CLI commands** – add new CLI entry points for MississippiFM pretraining and evaluation while preserving the existing logging and test infrastructure.

Is MississippiFM a good idea?

The idea is promising. The current pipeline demonstrates that self-supervised learning on NAIP imagery can significantly improve downstream segmentation and detection performance. MississippiFM builds on this success by exploiting known spatial overlap to provide dense supervision, potentially learning more transferable features than conventional SSL. The planned benchmark evaluates representation quality across diverse remote-sensing tasks and geographies, which could broaden the impact beyond poultry-house detection. The main risks are increased complexity and computational demands: constructing the adjacency index and training a ViT-B model with multiple objectives on millions of tiles will require careful engineering and substantial compute resources. Given the existing infrastructure and the clear research motivation, the project appears to be a sound and innovative extension of the current work.

Prompts for generating the MississippiFM codebase

I have compiled a set of detailed prompts that outline the new modules, scripts, and integration steps needed to implement MississippiFM. These prompts cover adjacency index creation, an overlap-aware dataset class, the DINOv3 encoder with multiple heads, the training loop with four objectives, evaluation scripts for multiple tasks, CLI integration, and high-level guidance. You can access them here:

Prompts for Generating the MississippiFM Codebase

The MississippiFM project extends the existing `poultry_geo` package by introducing overlap-aware self-supervised learning on sub-meter aerial imagery, integrating a DINOv3 Vision Transformer backbone, and expanding the evaluation to multiple geospatial tasks. The following prompts are meant to guide the generation of a comprehensive codebase that implements this vision.

1. Adjacency Index and Tile Metadata

Write a Python script called `create_adjacency_index.py` that constructs a lightweight adjacency index for existing NAIP tiles. The script should:

- Accept a root directory containing millions of 512×512 JPEG tiles where each filename encodes the original raster name and the x/y tile offsets.
- Parse filenames to recover the raster identifier and the tile's starting row and column indices in pixel space.
- Identify all pairs of tiles that overlap by 50 % in both x and y directions. For each overlapping pair, compute the pixel offsets that map the shared sub-region from the first tile into the second.
- Output a simple CSV file where each record contains: `tile_a`, `tile_b`, `row_offset`, `col_offset`, `overlap_x1`, `overlap_y1`, `overlap_x2`, `overlap_y2`. Use integer pixel indices throughout. Do not reslice the source imagery; use only filename metadata.
- Provide command-line arguments for the input directory, the overlap ratio, and the output path.

2. Overlap-Aware Dataset Class

Implement a PyTorch `Dataset` class named `OverlapAwareNAIPDataset` that extends the existing `NAIPDataset` to support overlap-aware self-supervised learning. This dataset should:

- Read the adjacency index created above and load tile pairs on demand. In each epoch, sample three types of pairs: 50 % overlapping adjacent tiles, 25 % two independent augmentations of the same tile, and 25 % independent random tiles.
- For overlapping pairs, return both images along with tensors describing the overlap mask coordinates. Compute and return the set of patch indices that correspond to the shared geographic region given the 16×16 patch grid size of ViT-B/16.
- For each sample, apply synchronized random augmentations (color jitter, flips, crops) while maintaining the integrity of the pixel offsets so that the overlap mapping remains accurate.
- Support optional high-resolution adaptation: when `high_res=True`, produce 512×512 crops with recorded crop origins; otherwise use random 256×256 or 320×320 views.

- Include methods to preload tile metadata and to switch between frozen and train splits. Do not allow adjacent tiles from the same raster to appear in both train and validation partitions.

3. DINOv3-Based MississippiFM Model

Create a PyTorch module called `MississippiFMEncoder` that wraps a DINOv3 ViT-B/16 backbone (from `timm` or a similar library) and adds heads for the four training objectives. The module should:

- Expose a method `forward_views` that accepts a batch of global and local views, returns the CLS token embeddings and patch embeddings, and projects them through separate MLP heads for global distillation and masked patch prediction.
- Include an MLP projection head used exclusively for the geographic overlap alignment loss. This head should map patch embeddings into a lower-dimensional space for cosine-similarity alignment.
- Implement a Gram anchoring mechanism: during initialization, load the weights of a pretrained DINOv3 model and freeze a copy as the anchor. During training, compute the Gram matrices of current patch embeddings and anchor embeddings and penalize deviations via a mean-squared-error loss scaled by a tunable weight.
- Provide methods to enable or disable LoRA adapters and NF4 quantization for parameter-efficient fine-tuning.

4. Training Loop with Multiple Objectives

Write a training script (`train_mississippifm.py`) that orchestrates the joint optimization of the four objectives: global self-distillation, masked patch prediction, geographic overlap alignment, and Gram anchoring. The script should:

- Accept configuration parameters from a YAML file specifying learning rates, batch sizes, loss weights, and the proportions of overlapping vs. independent pairs.
- Use the `OverlapAwareNAIPDataset` to produce batches containing both overlapping and independent pairs. For each batch, compute the four losses: DINO-style global distillation between different views, iBOT-style masked patch prediction, cosine alignment between corresponding patches of overlapping tiles, and Gram anchoring against the frozen backbone.
- Perform two-stage training: first train on random 256×256 or 320×320 crops for the majority of epochs, then adapt to full 512×512 crops for the final 10 %–15 % of training. Use mixed precision (`bf16` or `fp16`) and checkpoint optimizer and scheduler states after every epoch to allow resumption.
- Log training and validation metrics (loss components, learning rate, gradient norms) to TensorBoard and write a command log similar to the existing CLI wrappers.

- Save checkpoints at regular intervals and provide a `--resume` option to continue training from the last checkpoint. Include code to evaluate frozen representation quality on downstream validation probes (e.g., poultry segmentation) to select the best checkpoint.

5. Downstream Evaluation and Probes

Extend the current evaluation suite to support the diagnostic probes and cross-task benchmarks described in the MississippiFM specification. Implement:

- A script `evaluate_probes.py` that loads a frozen MississippiFM encoder, attaches simple linear or shallow heads, and measures performance on: agricultural pattern segmentation (Agriculture-Vision), settlement context classification (OpenEarthMap or similar), building segmentation (Inria), oriented object detection (DOTA), and poultry-house segmentation (Alabama/Georgia). Use standard metrics like mIoU, Dice, AP50:95, and boundary F1.
- Utilities to run zero-shot geographic transfer evaluations: apply Alabama-trained thresholds to Georgia data without retraining and compute performance drops. Support optional few-shot fine-tuning on a small set of Georgia farms.
- Diagnostic scripts that compute high-level spatial probes: building density regression, road-building relationship classification, field structure prediction, texture retrieval, and context augmentation experiments. These should operate on frozen features and report metrics like MAE, RMSE, recall@k, and F1.
- A module `geographic_benchmark.py` that orchestrates held-out county splits, different ground sampling distances, and temporal splits to produce the strict geographic and scale-shift benchmark results.

6. CLI Integration and Packaging

Update the existing `poultry_geo` CLI to include commands for MississippiFM pretraining and evaluation. Add a `mississippi-pretrain` command that wraps `train_mississippifm.py`, accepting options for data directories, adjacency index, model size (`vit_s`, `vit_b`, `vit_l`), and loss weights. Add `mississippi-eval` commands that invoke the probe and benchmark scripts. Ensure that each command logs arguments, supports resumption, and writes outputs to predictable directory structures under `trained_models/mississippifm/`.

Provide proper setup in `setup.py` or `pyproject.toml` to install dependencies (`timm`, `scikit-learn`, `openmmlab`, etc.) and expose console entry points. Add unit tests to the existing PyTest suite to cover adjacency index creation, overlap mapping logic, and loss computation correctness.

7. High-Level Project Guidance

Include a `README.md` describing the motivation for MississippiFM, the overlap-aware self-supervised approach, installation steps, data preparation, training commands, evaluation procedures, and citation guidelines. Emphasize that the existing six-million-tile corpus should not be regenerated and that the adjacency index provides the only additional metadata required for overlap training.



These prompts should help you or another developer systematically generate and assemble the complete codebase for the MississippiFM project.